

Architecture - Revision 3

1. Phase 1 Architecture

OpenClaw agent

- > local Node.js CLI: agent-follow-up
- > SQLite registry/event store
- > OpenClaw native cron job (isolated session or internal turn)
- > deterministic watchdog evaluation
- > OpenClaw-native message send emitter
- > evidence/logs/state updates

2. Scheduler Choice

Chosen scheduler: OpenClaw native cron.

Reason: it is persistent, gateway-integrated, already supported by the environment, and better aligned with the OpenClaw ecosystem than external crontab/systemd for this use case.

3. Registration Mechanism

agent-follow-up is defined in Phase 1 as a **local Node.js CLI tool** that lives in the project directory and is invoked by the OpenClaw agent via local command execution or wrapper integration.

Initial command set direction:

- agent-follow-up register ...
- agent-follow-up resolve ...
- agent-follow-up list

4. Notification Emitter (Gate Closed)

Phase 1 concrete emitter command:

```
openclaw message send --channel telegram --target <telegram-id> --message "..."
```

This command is the baseline notification path. If emission fails, the failure is recorded in task events and surfaced as unverified delivery.

5. Observation Strategy

Task Class	Phase 1 Support t	Observation Source
Process-backed with owned marker/sentinel path	Yes	Marker file + optional PID as secondary evidence
Manual/marker-backed	Yes	Known completion/failure marker path
OpenClaw delegated session without explicit marker	No (Phase 2)	Unsupported in Phase 1

Conditional is removed from Phase 1 because it was too ambiguous.

6. Unknown-State Boundary

A task becomes **unknown** when its required observation source no longer confirms running, no terminal outcome is proven, and that condition persists for **2 watchdog cycles by default**.